



RSA KEY ANALYSIS & VULNERABILITY TESTING

in SSL / TLS Certificates

PROJECT TEAM

Bhumika Srinivas Teli

1BM24CS072

Bhumika U

1BM24CS073

Chinthan G Upadhya

1BM24CS084

Research Basis:

"Mining Your Ps and Qs: Detection of Widespread Weak Keys" — N. Heninger et al., USENIX 2012

Submitted: 2nd June 2026

BMS COLLEGE OF ENGINEERING

Dept. of Computer Science & Engineering

4th Semester | June 2026

Introduction

Why RSA Security in TLS Matters

01 Cryptography — The Foundation of Digital Security

Cryptography transforms readable data (plaintext) into an unreadable

format (ciphertext) using mathematical algorithms.

- ▶ Symmetric: Same key for encryption and decryption (AES, DES).
- ▶ Asymmetric: Separate public/private key pairs (RSA, ECC).
- ▶ RSA is one of the most widely deployed asymmetric systems.

02 RSA in TLS/SSL — Securing the Web

TLS (Transport Layer Security) is the backbone of HTTPS and secure

communications across the internet.

- ▶ RSA is used to authenticate servers via digital certificates.
- ▶ RSA public keys within X.509 certificates establish trust chains.
- ▶ Billions of encrypted sessions rely on RSA keypairs daily.

03 Why Weak RSA Keys Are a Critical Concern

Not all RSA keys are generated with sufficient entropy or randomness.

- ▶ Poor random number generators (PRNGs) can produce predictable primes.
- ▶ IoT devices and embedded systems are especially vulnerable.
- ▶ A broken RSA key exposes the entire encrypted session to attackers.

RSA Fundamentals

Mathematical basis of the RSA cryptosystem

Key Generation Process

Step 1

Choose two distinct large primes p and q

Step 2

Compute modulus: $n = p \times q$

Step 3

Compute Euler's totient: $\phi(n) = (p-1)(q-1)$

Step 4

Choose public exponent e such that $\gcd(e, \phi(n)) = 1$

Step 5

Compute private exponent d : $d \cdot e \equiv 1 \pmod{\phi(n)}$

Public Key: (n, e) **Private Key:** (n, d)

Encryption

Sender uses recipient's PUBLIC key (n, e)

$$c \equiv m^e \pmod{n}$$

c = ciphertext, m = original message (plaintext)

Ciphertext c is sent over the public channel.

Decryption

Recipient uses their PRIVATE key (n, d)

$$m \equiv c^d \pmod{n}$$

$d \cdot e \equiv 1 \pmod{\phi(n)}$ ensures correct recovery of m

Security relies on the hardness of factoring large n into p and q .

TLS & Digital Certificates

How RSA enables secure communication on the internet

TLS HANDSHAKE PROTOCOL (demonstrated using Wireshark)



Inside an X.509 Certificate

Subject (CN)	Domain name of the server
Issuer	Certificate Authority (CA) identity
Validity	Not-Before and Not-After dates
Public Key	RSA modulus (n) and public exponent (e)
Signature Algo	SHA256withRSA, SHA384withRSA, etc.
Extensions	SANs, Key Usage, Extended Key Usage

RSA in Authentication & Digital Signatures



Server Identity Proof

Server signs a message with its RSA private key d . Client verifies the signature using the certificate's public key (n, e) .



Session Key Establishment

Client encrypts a pre-master secret with the server's public key. Only the server's private key can decrypt it.



Certificate Chain of Trust

CAs sign server certificates using their own private keys, creating a verifiable trust hierarchy.

Project Objective

Scope and goals of this academic research project

Based on the landmark research paper: "Mining Your Ps and Qs" — N. Heninger et al.

01

Large-Scale Key Collection

- ▶ Connect to 10,000 HTTPS web servers programmatically.
- ▶ Extract RSA public key parameters: modulus (n) and exponent (e).
- ▶ Store all extracted key data into a structured MySQL database.
- ▶ Target a diverse, real-world dataset of live certificates.

02

Vulnerability Analysis

- ▶ Apply three major mathematical cryptanalysis techniques.
- ▶ Batch GCD — detect shared prime factors across all key pairs.
- ▶ Fermat's Factorization — detect primes that are too close.
- ▶ Wiener's Attack — detect dangerously small private exponents (d).

03

Controlled Academic Demonstration

- ▶ All operations restricted strictly to public certificate data.
- ▶ No unauthorized access or intrusion into any server.
- ▶ Demonstrate attack algorithms on synthetically vulnerable keys.
- ▶ Quantify how modern real-world RSA keys stand up to these attacks.

Data Collection Methodology

How 10,000 RSA public keys were extracted and stored



Java — Key Extraction Module

```
// Core Java Packages Used:
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

import javax.net.ssl.SSLSocket;
import javax.net.ssl.SSLSocketFactory;
import javax.net.ssl.SSLSession;

import java.security.cert.X509Certificate;
import java.security.interfaces.RSAPublicKey;
import java.math.BigInteger;

import java.io.BufferedReader;
import java.io.FileReader;
```

Python — Batch GCD Analysis Module

```
# Python Packages Used:
import pandas as pd
import mysql.connector
import matplotlib.pyplot as plt
from math import gcd
from tqdm import tqdm

# Purpose of Each Package:
# pandas          → Load/analyse key dataset
# mysql.connector → Fetch data from MySQL DB
# matplotlib      → Visualise key distribution
# math.gcd        → Compute pairwise GCD
# tqdm            → Progress bar for batch ops
```

Batch GCD Attack

Exploiting shared prime factors across multiple RSA moduli

Core Concept

If two RSA moduli n_1 and n_2 share a prime factor p due to a flawed random number generator, then $\gcd(n_1, n_2) = p$ directly reveals the secret prime — completely breaking both private keys.

Mathematics

$$\gcd(n_1, n_2) = p \rightarrow q_1 = n_1/p$$

If result $\neq 1$, private key d is immediately compromised

Efficient over millions of keys using product-tree algorithm ($O(N \log^2 N)$).

ATTACK WORKFLOW

Collect

Gather all RSA moduli n from certificate database into a list.

Pairwise GCD

Compute $\gcd(n_i, n_j)$ for every pair — or use product tree for efficiency.

Factor Check

Any result $g > 1$ reveals a shared prime $p = g$, $q = n/g$.

Key Recovery

Recompute private exponent d using $\phi(n) = (p-1)(q-1)$.

✓ Advantage: Catastrophically effective against entropy-poor key generators.

✗ Limitation: No vulnerabilities found in our 4,272-key modern dataset.

Fermat's Factorization Attack

Exploiting RSA moduli where primes p and q are too close

Underlying Principle

If primes p and q are close to each other, then $\sqrt{n} \approx (p+q)/2$, meaning n can be expressed as a difference of two perfect squares: $n = a^2 - b^2 = (a+b)(a-b) = p \times q$

Vulnerability Requirement

The attack requires $|p - q|$ to be very small. A weak key generator that produces primes close in magnitude makes this attack computationally trivial — even for 2048-bit keys.

MATHEMATICAL STEPS

$$a = \lceil \sqrt{n} \rceil$$

Start with the ceiling of the square root of n .

$$b^2 = a^2 - n$$

Compute b^2 and check if it is a perfect square.

If b^2 not square

Increment a by 1 and repeat the computation.

Once $b^2 = \text{perfect square}$

Then $p = a - b$ and $q = a + b$.

Example: $n = 77 = 7 \times 11 \rightarrow \sqrt{77} \approx 8.77 \rightarrow a = 9 \rightarrow b^2 = 81 - 77 = 4 \rightarrow b = 2 \rightarrow p = 7, q = 11$

Wiener's Attack

Breaking RSA when a small private exponent is used for performance

⚠ Vulnerability: Choosing a small private exponent d (to speed up RSA decryption) makes the private key directly recoverable from the public key using continued fraction expansion.

Condition for Vulnerability

$$d < \frac{1}{3} \cdot n^{(1/4)}$$

Wiener's bound — d must satisfy this for the attack to succeed

When this holds, the continued fraction expansion of e/n yields d .

Continued Fraction Expansion of e/n

e/n can be approximated by a continued fraction $[a_0; a_1, a_2, \dots]$. Each convergent k/d of this fraction is a candidate for the private exponent. The correct d is found when $d \cdot e \equiv 1 \pmod{\phi(n)}$ is satisfied.

ATTACK PROCEDURE

- 1 Compute the continued fraction expansion of e/n .
- 2 Generate each convergent fraction p_k/q_k of the expansion.
- 3 For each convergent, test if q_k is a valid private key candidate.
- 4 Verify by checking: $e \cdot q_k - 1$ is divisible by a valid $\phi(n)$.

✓ Advantage: Polynomial-time attack — extremely fast once condition is met.

✗ Limitation: Modern systems use large d — attack is no longer practical today.

Implementation Details

System architecture, tools, and technology stack

Java 17+

Python 3.x

MySQL

BouncyCastle

BigInteger

Wireshark

SYSTEM WORKFLOW

Website
List (CSV)

Java SSL
Connector

X.509 Cert
Parser

MySQL
Database

Python
Batch GCD

Vulnerability
Report

Java — TLS Certificate Extraction

- ▶ Reads a list of 10,000 domains from a CSV file.
- ▶ Opens an SSLSocket connection on port 443 to each domain.
- ▶ Retrieves the server's X.509 certificate chain from the SSLSession.
- ▶ Casts the public key to RSAPublicKey to extract:
 - Modulus (n) via `.getModulus()` [BigInteger]
 - Exponent (e) via `.getPublicExponent()` [BigInteger]
- ▶ Inserts (domain, n , e , key_size, signature_algorithm) into MySQL.
- ▶ Uses PreparedStatement for safe, efficient batch inserts.

Python — Batch GCD Analysis

- ▶ Loads all (n , e) pairs from MySQL using `mysql.connector`.
- ▶ Converts data to a pandas DataFrame for efficient manipulation.
- ▶ Iterates all unique key pairs (n_i , n_j) using nested loops.
- ▶ Computes $\text{gcd}(n_i, n_j)$ using `math.gcd` (arbitrary precision).
- ▶ Flags any pair where gcd result is neither 1 nor n_i itself.
- ▶ `tqdm` provides a real-time progress bar for the lengthy batch job.
- ▶ `matplotlib` generates key-size and signature distribution charts.
- ▶ Results and statistics are exported as a final analysis report.

Results & Findings

Analysis of 10,000 real-world SSL/TLS certificates

10,000

Websites Targeted

4,538

Valid RSA Certs Found

65,537

Universal Exponent (e)

0

Shared-Prime Vuln. Found

Key Size Distribution		
Key Size	Count	Percentage
2048-bit	4,229	93.2%
4096-bit	281	6.2%
3072-bit	28	0.6%
TOTAL	4,538	100%

Signature Algorithm Distribution		
Algorithm	Count	% Share
SHA256withRSA	4,439	97.8%
SHA384withRSA	95	2.1%
SHA384withECDSA	2	0.04%
SHA512withRSA	2	0.04%

✓ KEY FINDING: Zero shared-prime vulnerabilities detected in 4,272 RSA key pairs tested — modern TLS implementations demonstrate robust entropy and secure key generation

Security Implications

What weak RSA key generation means for real-world security



Risks of Weak Key Generation

- ▶ Entropy starvation occurs in headless servers, IoT devices, and cloud VMs during early boot when insufficient system randomness is available.
- ▶ Predictable PRNG seeds produce nearly identical prime factors across many devices.
- ▶ Millions of deployments from the same vendor/firmware may share prime factors.



Real-World Consequences

- ▶ A compromised RSA private key allows full decryption of all past and future TLS sessions.
- ▶ Man-in-the-middle attacks become trivial — attackers can impersonate any affected server.
- ▶ Digital signatures can be forged, undermining authentication and certificate validity.
- ▶ Heninger et al. found 0.2% of all internet RSA keys vulnerable — thousands of live servers.



Best Practices & Defences

- ▶ Use a minimum of 2048-bit RSA keys — prefer 4096-bit for long-lived certificates.
- ▶ Ensure a high-entropy random number generator (e.g., `/dev/urandom`, `RDRAND`).
- ▶ Consider migrating to Elliptic Curve Cryptography (ECC) — smaller, faster, equally secure.
- ▶ Regularly audit deployed certificates and revoke/reissue if entropy is suspect.

Ethical Considerations

Research conducted with full legal and ethical compliance

Public Data Only



This research exclusively utilized publicly accessible SSL/TLS certificate information that any browser or client can retrieve freely via standard HTTPS connections. No private keys, session data, or server-side resources were accessed or targeted.

No Unauthorized Access



All network connections were standard TLS handshakes, identical to what a web browser performs. No server was compromised, exploited, or subjected to attacks of any kind. The project strictly adheres to computer misuse laws and institutional research guidelines.

Educational & Research Purpose



This project was conducted solely to understand mathematical vulnerabilities theoretically and to quantify the security posture of modern web certificates as an academic exercise. The findings contribute to the broader understanding of practical RSA security in the wild.

Responsible Disclosure Policy



Had any live, exploitable vulnerabilities been discovered, the project team was committed to responsibly disclosing them to the affected certificate holder and relevant Certificate Authorities before publishing any findings, per standard security research ethics.

Conclusion

Summary of findings and lessons from this research project

KEY FINDING

Analysis of 4,538 live SSL/TLS certificates from 10,000 websites reveals that modern RSA implementations demonstrate strong entropy — zero shared-prime vulnerabilities were found, validating the effectiveness of current cryptographic standards.

SUMMARY OF FINDINGS

Mathematical Robustness

All 4,229 two-thousand-forty-eight-bit and 281 four-thousand-ninety-six-bit keys analyzed showed no shared prime factors, indicating strong PRNG entropy in modern TLS deployments.

Universal Exponent Adoption

100% of analyzed certificates use $e = 65537$, confirming universal adherence to the industry-recommended public exponent that balances performance and security.

Signature Algorithm Maturity

SHA256withRSA dominates at 97.8%, demonstrating industry-wide migration away from deprecated MD5 and SHA-1 algorithms toward collision-resistant hashing.

Theoretical Attacks Remain Relevant

While modern certificates are robust, Batch GCD, Fermat, and Wiener attacks remain critical threats for constrained IoT devices with weak entropy sources.

References

Research papers, standards, and documentation

[1]

N. Heninger, Z. Durumeric, E. Wustrow, J. A. Halderman

"Mining Your Ps and Qs: Detection of Widespread Weak Keys in Network Devices"

USENIX Security Symposium, 2012 (Primary Research Paper)

[2]

M. J. Wiener

"Cryptanalysis of Short RSA Secret Exponents"

IEEE Transactions on Information Theory, Vol. 36, No. 3, 1990

[3]

D. Boneh, G. Durfee

"Cryptanalysis of RSA with Private Key d Less than $N^{0.292}$ "

EUROCRYPT 1999, LNCS Vol. 1592

[4]

R. L. Rivest, A. Shamir, L. Adleman

"A Method for Obtaining Digital Signatures and Public-Key Cryptosystems"

Communications of the ACM, Vol. 21, No. 2, 1978

[5]

IETF RFC 8446

"The Transport Layer Security (TLS) Protocol Version 1.3"

IETF, August 2018

[6]

IETF RFC 5246

"The Transport Layer Security (TLS) Protocol Version 1.2"

IETF, August 2008

[7]

RSA Laboratories

"PKCS #1 v2.2: RSA Cryptography Standard"

RSA Security LLC, 2012

[8]

W. Diffie, M. Hellman

"New Directions in Cryptography"

IEEE Transactions on Information Theory, Vol. 22, No. 6, 1976

Thank You

BMS College of Engineering | Dept. of Computer Science & Engineering

4th Semester | June 2026

Bhumika Srinivas Teli (1BM24CS072) | Bhumika U (1BM24CS073) | Chinthan G Upadhya (1BM24CS084)

"Security is not a product, but a process." — Bruce Schneier